

I. Implicit Conversion (Widening)

1. Determine which assignments are valid without using *casting* and explain why.

Exercise	Code (Valid or Invalid?)	Why? (Implicit or Error?)
1.	<pre>int i = 50; long l = i;</pre>	
2.	<pre>float f = 15.0f; double d = f;</pre>	
3.	<pre>short s = 5; int i = s;</pre>	
4.	<pre>char c = 'Z'; int i = c;</pre>	
5.	<pre>byte b = 10; short s = b;</pre>	
6.	<pre>double d = 25.5; float f = d;</pre>	

II. Explicit Conversion (Casting or Narrowing)

2. Use explicit *casting* to fix compilation errors and predict the potential loss of information.

Instruction: Correct the code in the "Initial Code" column so that it compiles successfully. Then, in the "Prediction" column, indicate what value will be lost or what data will be truncated.

Exercise	Initial Code (Error)	Corrected Code (With Casting)	Result of	Prediction of Information Loss
7.	<pre>double d = 3.14159; int newType = d;</pre>	<pre>int newType = (int) d;</pre>	<pre>System.out.println(new Type); 3</pre>	Decimals are lost (, 14159).
8.	<pre>long l = 1000L; int</pre>			

- ```

newType =
1;

9. int i = 67;
 char
 newType =
 i;

10. int i =
 300; byte
 newType =
 i;

11. float f =
 123.456f;
 long
 newType =
 f;

```

### III. Integrated Challenge (Mixed Conversions)

3. Apply the rules of implicit promotion in arithmetic operations and the need for *casting* when assigning the result.

**Instruction:** Indicate the final data type of the result variable and whether explicit *casting* is needed to store the value.

| Exercise | Code                                                           | Expected Data Type of the Result (Ex: int, long, double) | Casting Needed (Yes/No)? |
|----------|----------------------------------------------------------------|----------------------------------------------------------|--------------------------|
| 12.      | <pre> int x = 5; double y = 2.0; double result = x / y; </pre> | double                                                   | No                       |
| 13.      | <pre> short a = 10; short b = 2; short result = a * b; </pre>  |                                                          |                          |
| 14.      | <pre> long l = 100L; int i = 50; int result = l + i; </pre>    |                                                          |                          |

**15.**     `char c = 'a'; int i =  
1; int result = c +  
i;`

**16.**     `byte b1 = 5; byte b2  
= 10; byte result =  
b1 + b2;`